



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*
Example Logic: IF `food_here` THEN `suck`; IF `wall_ahead` THEN `turn_left`;
ELSE `move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF `wall_ahead` AND `left_is_visited` THEN `turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A Table-Driven Agent stores the correct action for every possible percept sequence. In complex environments such as Chess, the number of possible percept sequences is extremely large, making the lookup table impossibly large to create and store. As the agent's lifetime increases, the percept sequence becomes longer, causing the table size to grow exponentially. Therefore, a mathematically perfect Table-Driven Agent is impractical for real-world problems.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

```
if percept["food_here"]:  
    return "Up"  
  
elif percept["toxin_here"]:  
    return "Right"  
  
elif percept["wall_here"]:  
    return "Left"  
  
else:  
    return random.choice(["Up", "Down", "Left", "Right"])
```

Each condition checks the current percept, and each action returns the movement to perform. The agent makes decisions using only the current percept without storing any previous information, which matches the Simple Reflex Agent architecture.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The Simple Reflex Agent only uses the current percept and does not remember previous states. Because the environment is partially observable, the agent cannot see the complete environment or remember where it has already been. When it encounters the same local percept repeatedly (for example, near walls or obstacles), it applies the same Condition–Action Rule every time. Since it has no percept history or internal memory, it cannot recognize that it is repeating the same actions, which can cause it to move in loops or become trapped.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

In my ModelBasedAgent, the Sensor Model is implemented using the `get_percept()` method. The agent receives percepts such as `agent_pos`, `food_here`, `toxin_here`, `wall_here`, and `collision`. These percepts are used to update the agent's internal memory (`visited_cells` and `last_action`).

The Transition Model is represented by updating the internal state after each percept. The agent records its current position in `visited_cells` and remembers the previous action using `last_action`. When the agent detects that it has already visited a location, it chooses an alternative action instead of repeating the same behavior. This allows the Model-Based Agent to use memory to make better decisions than the Simple Reflex Agent, although the environment itself still determines the actual movement through `execute_action()`.

Finalize and Lock Lab 02 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING